

A COMPLETE BEGINNER'S GUIDE

SOLID Design Principles in C++

Five principles for code that is easy to change, test and extend



- Each principle: definition, plain-English meaning and a real-life analogy
- A broken "before" program and a fixed "after" program, both compiled and run
- UML class diagrams of every violation and every fix
- A full refactoring project that applies all five principles together
- When NOT to apply SOLID, interview questions, exercises, cheat sheet

Contents

1. Why Design Principles Matter	5
1.1 Signs of rotting design	5
1.2 Coupling and cohesion	5
1.3 Where SOLID comes from	6
1.4 The five principles at a glance	6
1.5 How SOLID builds on the four pillars	7
2. Single Responsibility Principle	8
2.1 What it means	8
2.2 The problem	8
2.3 The fix: one class per reason to change	10
2.4 How to spot SRP violations	11
2.5 Common misunderstandings	12
3. Open/Closed Principle	13
3.1 What it means	13
3.2 The problem	14
3.3 The fix: depend on an abstraction, extend with new classes	15
3.4 Ways to achieve OCP in C++	16
3.5 You cannot be closed against everything	17
3.6 How to spot OCP violations	17
4. Liskov Substitution Principle	18
4.1 What it means	18
4.2 The classic problem: Square and Rectangle	18
4.3 The fix: model behaviour, not words	19
4.4 Second example: birds that cannot fly	21
4.5 The rules of substitutability	23
4.6 How to spot LSP violations	23
5. Interface Segregation Principle	25
5.1 What it means	25
5.2 The problem	26
5.3 The fix: small role interfaces	28
5.4 How to spot ISP violations	29
6. Dependency Inversion Principle	30
6.1 What it means	30

6.2 The problem	31
6.3 The fix: depend on an abstraction, inject the detail	32
6.4 DIP, dependency injection and inversion of control	33
6.5 DIP with templates	34
6.6 How to spot DIP violations	34
7. Putting It All Together: Refactoring an Order System	36
7.1 Before	36
7.2 After	38
7.3 Where each principle appears	40
8. Using SOLID Wisely	41
8.1 Signs you have gone too far	41
8.2 Principles that balance SOLID	41
8.3 SOLID and design patterns	41
9. Interview Questions and Answers	43
10. Practice Exercises	45
Design challenges	47
11. Cheat Sheet and Glossary	48
Glossary	48

How to use this guide

SOLID is a set of five design principles that help you write object-oriented code that stays easy to change as a project grows. You need to be comfortable with C++ classes, inheritance, virtual functions and abstract classes (the four pillars of OOP). Reading UML class diagrams helps too; every diagram here uses standard UML notation.

Every principle chapter follows the same pattern, so you always know where you are:

1. **Definition:** the principle in its well-known form, then in plain English.
2. **Analogy:** the same idea in everyday life.
3. **The problem:** a realistic program that breaks the principle, with its real output, and an explanation of what goes wrong.
4. **The fix:** the same program redesigned to follow the principle, with output and a before/after UML diagram.
5. **Spotting violations, common mistakes and a recap.**

Tip

Run both versions of every program. Then try to add a new feature to each one (a new discount type, a new message channel). Feeling how much harder the "before" version is to change is the fastest way to understand why SOLID matters.

Compile with: `g++ -std=c++17 file.cpp -o app` then run `./app`.

Each principle has its own colour throughout the guide:

Letter	Principle	Colour	One-line idea
S	Single Responsibility	Blue	A class should have one reason to change
O	Open/Closed	Green	Add new behaviour without editing tested code
L	Liskov Substitution	Amber	Subclasses must keep the base class's promises
I	Interface Segregation	Purple	Prefer several small interfaces to one big one
D	Dependency Inversion	Teal	Depend on abstractions, not concrete details

1. Why Design Principles Matter

Getting a program to **work** is only the first step. Real software lives for years and changes constantly: new features, new rules, new devices, new team members. The real question is: **how expensive is the next change?** Good design keeps that cost low. Poor design makes every change slower and riskier than the last, until the team is afraid to touch the code.

1.1 Signs of rotting design

Robert C. Martin described the symptoms of badly designed code, often called **design smells**. You have probably felt some of them already:

Smell	What it feels like	Example
Rigidity	Every change forces many other changes	Adding a discount type means editing five files
Fragility	Changing one thing breaks something unrelated	Fixing the invoice layout breaks tax calculation
Immobility	Useful code cannot be reused because it is tangled with other code	Cannot reuse the pricing logic without dragging in the database code
Viscosity	Doing it the right way is harder than hacking it	Easier to add another <code>if</code> than to design properly
Needless complexity	Structure that does not pay for itself	Five interfaces for a 50-line program

SOLID is a set of tools for preventing the first four smells. The last one is a warning that SOLID itself can be overdone (Chapter 8).

1.2 Coupling and cohesion

Almost every design principle aims at two goals:

- **High cohesion**: everything inside a class belongs together and serves one purpose. A cohesive class is easy to name, understand and test.
- **Low coupling**: classes depend on each other as little as possible, and only through clear, stable interfaces. A change in one class then rarely forces changes in others.

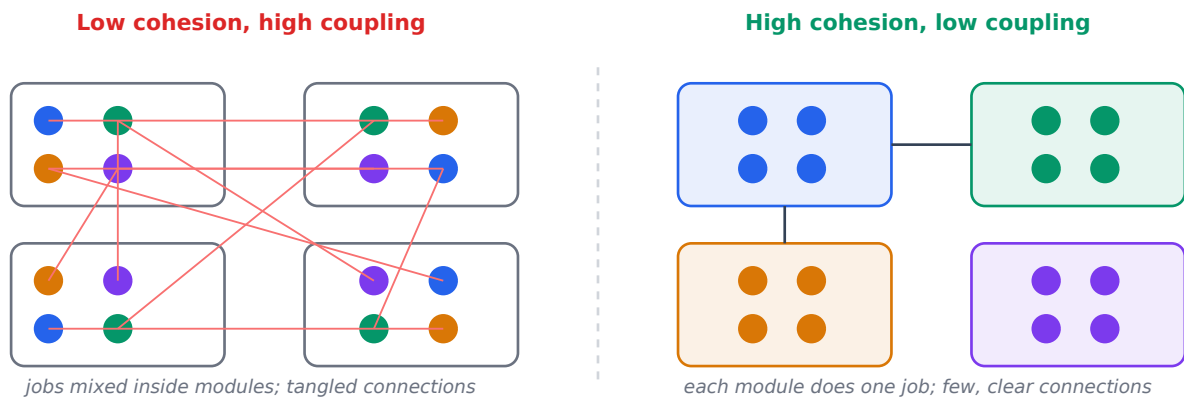


Figure 1.1 Each colour is a different responsibility. Good design groups them and connects modules sparingly.

1.3 Where SOLID comes from

The five principles were collected and popularised by **Robert C. Martin** ("Uncle Bob"), mainly in his 2000 paper *Design Principles and Design Patterns* and his later books. **Michael Feathers** noticed that their initials spell SOLID, and the name stuck. The ideas themselves are older: the Open/Closed Principle comes from **Bertrand Meyer** (1988), and the Liskov Substitution Principle from **Barbara Liskov**, who presented it in 1987 and later received the Turing Award for her work on data abstraction.

1.4 The five principles at a glance

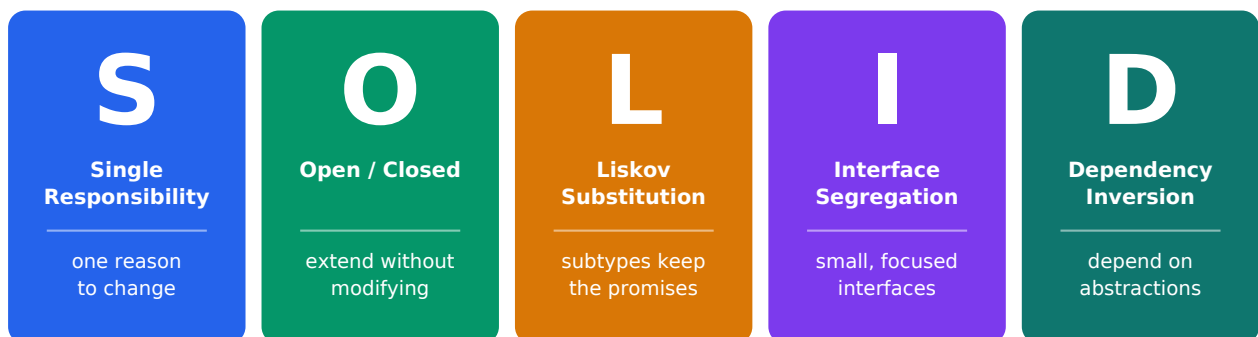


Figure 1.2 The SOLID principles.

Principle	In one sentence	Main problem it prevents
Single Responsibility	A class should have only one reason to change.	Unrelated changes colliding in the same class
Open/Closed	Extend behaviour by adding code, not by editing existing code.	Growing if/else chains; re-testing old code for every feature
Liskov Substitution	A subclass must work anywhere its base class works.	Inheritance that compiles but breaks callers at run time
Interface Segregation	Do not force a class to depend on methods it does not use.	Fat interfaces full of empty or throwing methods

Principle	In one sentence	Main problem it prevents
Dependency Inversion	High-level logic should depend on abstractions, not on concrete details.	Business logic welded to databases, files and frameworks

1.5 How SOLID builds on the four pillars

Principle	Pillars it relies on
SRP	Encapsulation : each class guards one cohesive set of data and behaviour
OCP	Abstraction + polymorphism : new behaviour plugs in behind a stable interface
LSP	Inheritance , used correctly: an is-a relationship must hold for behaviour, not just in name
ISP	Abstraction : interfaces shaped around what each client actually needs
DIP	Abstraction + polymorphism : depend on interfaces, inject the details

Key takeaways

- Good design is about keeping the cost of future change low.
- Aim for high cohesion inside classes and low coupling between them.
- SOLID = five principles that work together to achieve that.

2. Single Responsibility Principle

"A class should have one, and only one, reason to change."

Robert C. Martin. In *Clean Architecture* (2017) he restated it as: a module should be responsible to one, and only one, actor.

2.1 What it means

"Responsibility" here does not mean "one function" or "one thing". It means **one reason to change**, and reasons to change come from **people**: a team, a department, a stakeholder. If two different groups of people could each ask you to change the same class for unrelated reasons, that class has two responsibilities and should probably be split.

Real-life analogy

In a restaurant, the chef cooks, the waiter serves and the cashier handles bills. If the menu changes, only the chef's work changes. If the payment system changes, only the cashier is retrained.

Now imagine one person doing all three jobs. Every change, of any kind, lands on that one person, and a mistake in billing can delay the food.

2.2 The problem

This `Invoice` class works perfectly. Look at its output, then look at who might ask for each part of it to change:

C++ · `srp_bad.cpp`

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Item { string name; int qty; double price; };

// ONE class with FOUR different reasons to change
class Invoice {
    string customer;
    vector<Item> items;
public:
    Invoice(string c, vector<Item> i) : customer(c), items(i) {}

    // 1. Business rules          -> changes when the ACCOUNTS team changes tax rules
    double total() const {
        double sum = 0;
        for (const auto& it : items) sum += it.qty * it.price;
        return sum * 1.18;          // 18% GST
    }

    // 2. Presentation           -> changes when the UI team wants a new layout
    void print() const {
        cout << "INVOICE for " << customer << "\n";
    }
};
```

C++ · srp_bad.cpp

```
for (const auto& it : items)
    cout << " " << it.name << " x" << it.qty << "\n";
cout << " Total (incl. GST): Rs " << total() << "\n";
}

// 3. Persistence -> changes when OPS moves from files to a database
void saveToFile(const string& path) const {
    ofstream f(path);
    f << customer << ", " << total() << "\n";
    cout << "Saved to " << path << "\n";
}

// 4. Notification -> changes when MARKETING switches email provider
void emailTo(const string& address) const {
    cout << "Emailing invoice to " << address << "\n";
}

};

int main() {
    Invoice inv("Riya", {"Pen", 2, 50}, {"Notebook", 3, 120});
    inv.print();
    inv.saveToFile("invoice.csv");
    inv.emailTo("riya@example.com");
    return 0;
}
```

OUTPUT

```
INVOICE for Riya
  Pen x2
  Notebook x3
  Total (incl. GST): Rs 542.8
Saved to invoice.csv
Emailing invoice to riya@example.com
```

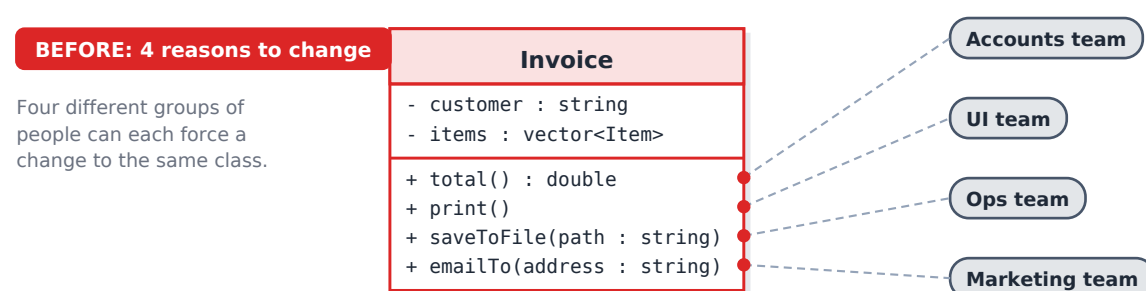


Figure 2.1 Four teams, four reasons to change, one class.

What goes wrong

- **Unrelated changes collide.** A designer changing `print()` must edit, recompile and re-test the same file that holds the tax rules.
- **Fragility.** A small mistake while editing the email code can break the invoice total, which is the part that involves money.
- **Hard to reuse.** Want to compute an invoice total in a mobile app? You also drag in file and email code the app does not need.
- **Hard to test.** Testing `total()` creates an object that also knows how to write files and send email.

2.3 The fix: one class per reason to change

Split the class along its reasons to change. **Invoice** keeps the data and the business rule (**total()**), and each other responsibility moves into its own small class that **uses** an **Invoice**:

C++ · **srp_good.cpp**

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Item { string name; int qty; double price; };

// Responsibility 1: invoice data + business rules (only reason: tax/pricing rules)
class Invoice {
    string customer;
    vector<Item> items;
public:
    Invoice(string c, vector<Item> i) : customer(c), items(i) {}
    const string& getCustomer() const { return customer; }
    const vector<Item>& getItems() const { return items; }
    double total() const {
        double sum = 0;
        for (const auto& it : items) sum += it.qty * it.price;
        return sum * 1.18;
    }
};

// Responsibility 2: presentation (only reason: layout changes)
class InvoicePrinter {
public:
    void print(const Invoice& inv) const {
        cout << "INVOICE for " << inv.getCustomer() << "\n";
        for (const auto& it : inv.getItems())
            cout << "    " << it.name << " x" << it.qty << "\n";
        cout << "    Total (incl. GST): Rs " << inv.total() << "\n";
    }
};

// Responsibility 3: persistence (only reason: storage changes)
class InvoiceRepository {
public:
    void save(const Invoice& inv, const string& path) const {
        ofstream f(path);
        f << inv.getCustomer() << "," << inv.total() << "\n";
        cout << "Saved to " << path << "\n";
    }
};

// Responsibility 4: notification (only reason: messaging changes)
class InvoiceMailer {
public:
    void send(const Invoice& inv, const string& address) const {
        cout << "Emailing invoice for " << inv.getCustomer()
              << " to " << address << "\n";
    }
};

int main() {
    Invoice inv("Riya", {{ "Pen", 2, 50 }, { "Notebook", 3, 120 }});
    InvoicePrinter().print(inv);
    InvoiceRepository().save(inv, "invoice.csv");
    InvoiceMailer().send(inv, "riya@example.com");
}
```

C++ · srp_good.cpp

```
    return 0;
}
```

OUTPUT

```
INVOICE for Riya
  Pen x2
  Notebook x3
  Total (incl. GST): Rs 542.8
Saved to invoice.csv
Emailing invoice for Riya to riya@example.com
```

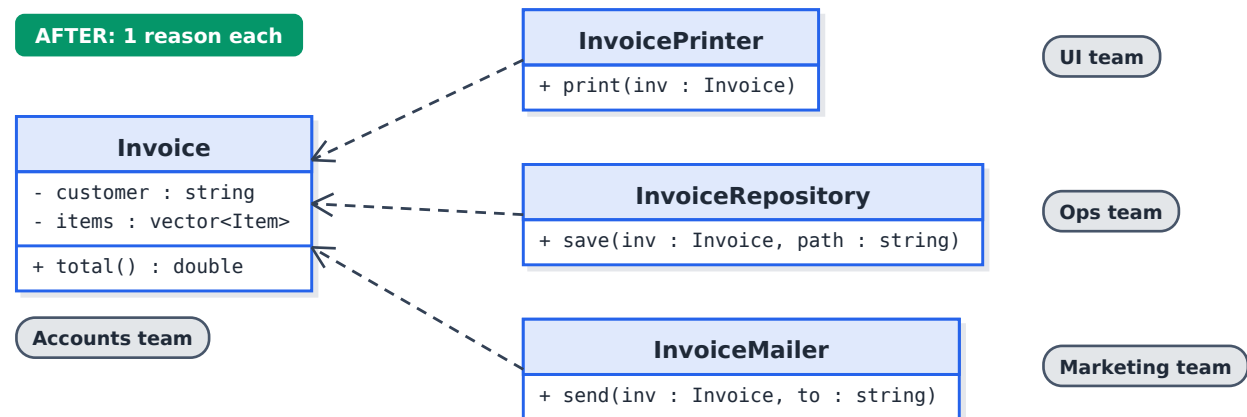


Figure 2.2 Each class now answers to exactly one group of people.

The output is identical: SRP changed the **structure**, not the behaviour. But now a layout change touches only **InvoicePrinter**, a move to a database touches only **InvoiceRepository**, and the tax rule lives in one place that nothing else edits.

2.4 How to spot SRP violations

Warning sign	Why it suggests multiple responsibilities
You describe the class with "and"	"It calculates totals and prints and saves" = three jobs
Very long class, many #includes	Files, networking, formatting and maths headers in one class usually mean mixed concerns
Groups of methods use different fields	Methods that share no data probably belong in different classes (low cohesion)
Generic names: Manager , Processor , Utils	Vague names often hide classes that do everything
Many unrelated reasons in your version history	The same file changed for UI, tax and database reasons
Hard to write a unit test	Testing one behaviour needs files, databases or networks set up

2.5 Common misunderstandings

- **"One responsibility = one method."** No. `Invoice` has several methods (getters, `total()`), all serving the same reason to change.
- **"Split everything as small as possible."** Over-splitting creates dozens of tiny classes and scatters related logic. Split along **real** reasons to change that you can name.
- **"SRP is only about classes."** It applies to functions, modules and services too. A function that validates, calculates and prints is also doing three jobs.

SRP recap

- A class should have one reason to change: it should answer to one actor.
- Find responsibilities by asking WHO would request each change.
- Split along those lines; keep related things together.
- Benefits: fewer collisions, safer changes, easier testing and reuse.

3. Open/Closed Principle

"Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification."

Bertrand Meyer, *Object-Oriented Software Construction* (1988); the polymorphism-based form used today was popularised by Robert C. Martin.

3.1 What it means

Open for extension: you can make the module do new things. **Closed for modification:** you do that **without editing its existing, tested source code**. It sounds contradictory, but abstraction makes it possible: the module is written against an interface, and new behaviour arrives as **new classes** that implement that interface.

Real-life analogy

A power socket is closed for modification (nobody rewires the wall to plug in a new device) and open for extension (any device with the right plug works: a lamp, a laptop charger, a device invented next year).

Smartphone apps work the same way: you add features by installing apps, not by opening the phone and rewiring it.

3.2 The problem

C++ · ocp_bad.cpp

```
#include <iostream>
#include <string>
using namespace std;

class DiscountCalculator {
public:
    // Every new customer type means EDITING this function again
    double discount(const string& customerType, double amount) const {
        if (customerType == "regular")
            return 0;
        else if (customerType == "premium")
            return amount * 0.10;
        else if (customerType == "vip")
            return amount * 0.20;
        // Marketing wants a "student" discount next month...
        // ...so someone must open this tested code and change it.
        return 0;
    }
};

int main() {
    DiscountCalculator calc;
    for (string type : {"regular", "premium", "vip", "student"})
        cout << type << ": discount Rs " << calc.discount(type, 2000) << "\n";
    return 0;
}
```

OUTPUT

```
regular: discount Rs 0
premium: discount Rs 200
vip: discount Rs 400
student: discount Rs 0
```

BEFORE: closed to extension

DiscountCalculator

+ discount(type : string, amount : double) : double

if (type == "regular") ...
else if (type == "premium") ...
else if (type == "vip") ...
else if (type == "student") ← EDIT!
every new type = modify + retest

Figure 3.1 The calculator has to be edited for every new customer type.

What goes wrong

- The output shows **student: discount Rs 0**: the new type silently gets the wrong result, because nobody has edited the function yet.
- To add it, someone must open working, tested code and change it, risking bugs in the existing branches.
- Type-based **if/else** chains like this tend to be **copied** across the program (discounts, labels, reports), so one new type means hunting down every copy.
- The class keeps growing forever.

3.3 The fix: depend on an abstraction, extend with new classes

C++ · ocp_good.cpp

```
#include <algorithm>
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

// The abstraction that keeps Checkout CLOSED for modification
class DiscountPolicy {
public:
    virtual ~DiscountPolicy() {}
    virtual double discount(double amount) const = 0;
    virtual string name() const = 0;
};

class RegularDiscount : public DiscountPolicy {
public:
    double discount(double) const override { return 0; }
    string name() const override { return "regular"; }
};

class PremiumDiscount : public DiscountPolicy {
public:
    double discount(double a) const override { return a * 0.10; }
    string name() const override { return "premium"; }
};

class VipDiscount : public DiscountPolicy {
public:
    double discount(double a) const override { return a * 0.20; }
    string name() const override { return "vip"; }
};

// ADDED LATER: a new class. No existing line of code was changed.
class StudentDiscount : public DiscountPolicy {
public:
    double discount(double a) const override { return min(a * 0.15, 250.0); }
    string name() const override { return "student"; }
};

// Written once, never edited again when new discounts appear
class Checkout {
public:
    double bill(double amount, const DiscountPolicy& policy) const {
        double d = policy.discount(amount);
        cout << policy.name() << ": discount Rs " << d
              << ", pay Rs " << amount - d << "\n";
        return amount - d;
    }
};

int main() {
    vector<unique_ptr<DiscountPolicy>> policies;
    policies.push_back(make_unique<RegularDiscount>());
    policies.push_back(make_unique<PremiumDiscount>());
    policies.push_back(make_unique<VipDiscount>());
    policies.push_back(make_unique<StudentDiscount>());

    Checkout checkout;
    for (const auto& p : policies)
```

C++ · ocp_good.cpp

```
    checkout.bill(2000, *p);  
    return 0;  
}
```

OUTPUT

```
regular: discount Rs 0, pay Rs 2000  
premium: discount Rs 200, pay Rs 1800  
vip: discount Rs 400, pay Rs 1600  
student: discount Rs 250, pay Rs 1750
```

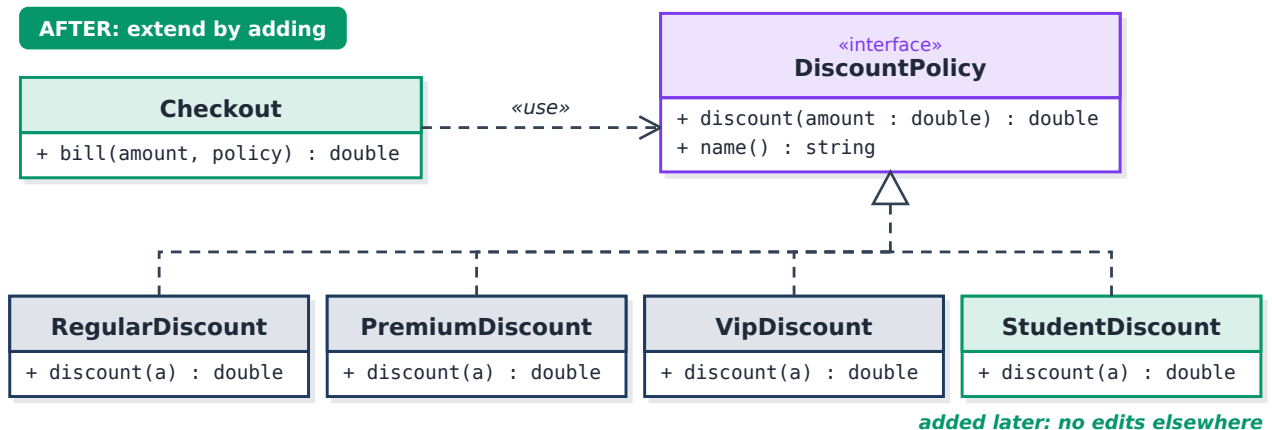


Figure 3.2 New discounts are new classes; Checkout and the existing policies never change.

StudentDiscount (15%, capped at Rs 250) was added by **writing one new class**. **Checkout**, **DiscountPolicy** and the three existing policies were not touched, so they do not even need re-testing. That is the Open/Closed Principle.

3.4 Ways to achieve OCP in C++

Technique	How	Best when
Virtual functions (runtime polymorphism)	Interface + subclasses, as above	Behaviour chosen at run time (user picks a payment type)
Templates (compile-time polymorphism)	Write the module as a template over a policy type	Behaviour fixed at compile time; performance-critical code
Function objects / <code>std::function</code>	Pass the varying behaviour in as a callable	Small pieces of behaviour, like a sort comparator
Plugins / configuration	Load new behaviour from data or modules	Large systems with third-party extensions

Note

`std::sort` is a great everyday example of OCP: it is closed (you never edit it) but open (pass your own comparator to sort in any order you like).

3.5 You cannot be closed against everything

No design is closed against every possible change. If you add abstraction for every conceivable variation, the code becomes needlessly complex. Instead, **close the module against the changes that are likely**: the ones requirements already mention, or that have already happened once. A good practical rule is to write simple code first and **introduce the abstraction the second or third time** that kind of change appears.

3.6 How to spot OCP violations

- `if/else if` or `switch` chains on a **type code** (`type == "vip"`).
- The same type check repeated in several places.
- Adding a feature always means editing the same central class.
- Use of `dynamic_cast` or `typeid` to decide what to do with an object.

OCP recap

- Add new behaviour by adding new code, not by editing old code.
- Write the stable part against an interface; put variations behind it.
- Replace type-switches with polymorphism.
- Close against likely changes, not every imaginable one.

4. Liskov Substitution Principle

Objects of a subclass must be usable wherever objects of the base class are expected, without breaking the program's correctness.

Paraphrasing Barbara Liskov (1987) and Liskov & Wing, *A Behavioral Notion of Subtyping* (1994).

4.1 What it means

Inheritance in C++ makes a `Square*` compile wherever a `Rectangle*` is expected. LSP says that is not enough: the subclass must also **behave** in the way callers of the base class expect. If code written for the base class has to check "is this actually a Square?" to work correctly, the hierarchy is wrong.

Real-life analogy

If a recipe needs "a battery", any battery of the right size should work. A battery that fits the slot but leaks, or only works upside down, technically fits but breaks the device.

A rubber duck **looks** like a duck, but if your code expects a duck that can fly and swim on its own, the rubber duck is not a valid substitute.

4.2 The classic problem: Square and Rectangle

In geometry, a square *is* a rectangle, so inheriting seems natural. Watch what happens to code written for rectangles:

C++ · `lsp_bad.cpp`

```
#include <iostream>
using namespace std;

class Rectangle {
protected:
    int w = 0, h = 0;
public:
    virtual ~Rectangle() {}
    virtual void setWidth(int x) { w = x; }
    virtual void setHeight(int y) { h = y; }
    int area() const { return w * h; }
};

// "A square IS-A rectangle" ... mathematically true, so let's inherit?
class Square : public Rectangle {
public:
    void setWidth(int x) override { w = h = x; } // keep sides equal
    void setHeight(int y) override { w = h = y; }
};

// Client code written for Rectangle, relying on its normal behaviour
void resize(Rectangle& r) {
    r.setWidth(5);
    r.setHeight(4);
    cout << "Expected area 20, got " << r.area();
}
```

C++ · lsp_bad.cpp

```
<< (r.area() == 20 ? " OK" : " BROKEN") << "\n";
}

int main() {
    Rectangle rect;
    Square square;
    resize(rect);
    resize(square);    // substituting a Square breaks the client
    return 0;
}
```

OUTPUT

```
Expected area 20, got 20    OK
Expected area 20, got 16    BROKEN
```

BEFORE

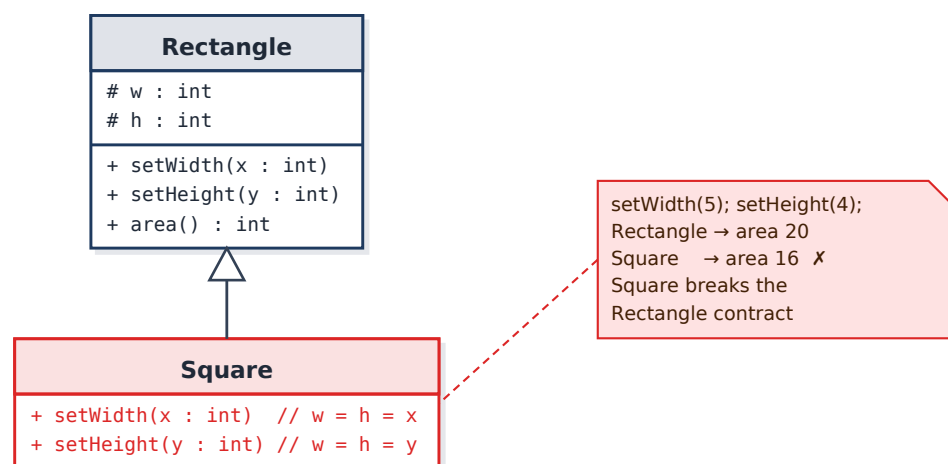


Figure 4.1 Square compiles as a Rectangle but breaks Rectangle's behaviour.

What goes wrong

`resize()` relies on a perfectly reasonable expectation of any rectangle: **setting the height does not change the width**. Square breaks that rule to keep its sides equal, so the same function gives 16 instead of 20.

The bug is not in `resize()` and not in `Square` alone; it is in the **inheritance relationship**. The compiler cannot catch it, so it shows up as a wrong answer at run time, possibly far away from where the classes were written.

4.3 The fix: model behaviour, not words

A mutable square does not behave like a mutable rectangle, so it should not inherit from one. Both can implement a common abstraction that promises only what **both** can honour:

C++ · lsp_good.cpp

```
#include <iostream>
#include <memory>
#include <vector>
using namespace std;

// Common abstraction that promises only what EVERY shape can honour
class Shape {
```

C++ · lsp_good.cpp

```
public:
    virtual ~Shape() {}
    virtual int area() const = 0;
};

class Rectangle : public Shape {
    int w, h;
public:
    Rectangle(int width, int height) : w(width), h(height) {}
    int area() const override { return w * h; }
};

class Square : public Shape {
    int side;
public:
    explicit Square(int s) : side(s) {}
    int area() const override { return side * side; }
};

int main() {
    vector<unique_ptr<Shape>> shapes;
    shapes.push_back(make_unique<Rectangle>(5, 4));
    shapes.push_back(make_unique<Square>(4));

    // Any Shape can be used here, and every one behaves as a Shape promises
    for (const auto& s : shapes)
        cout << "area = " << s->area() << "\n";
    return 0;
}
```

OUTPUT

```
area = 20
area = 16
```

AFTER

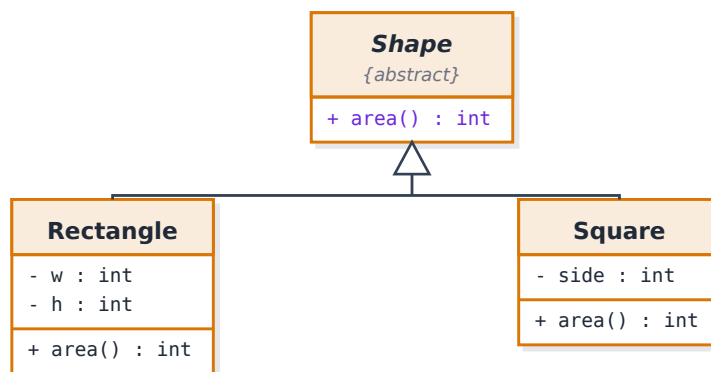


Figure 4.2 Rectangle and Square are siblings under Shape.

4.4 Second example: birds that cannot fly

C++ · birds_bad.cpp

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

class Bird {
public:
    virtual ~Bird() {}
    virtual string name() const = 0;
    virtual void fly() const { cout << name() << " is flying\n"; }
};

class Sparrow : public Bird {
public:
    string name() const override { return "Sparrow"; }
};

class Penguin : public Bird {
public:
    string name() const override { return "Penguin"; }
    void fly() const override { // cannot keep Bird's promise
        throw logic_error("Penguins can't fly!");
    }
};

void migrate(const vector<unique_ptr<Bird>>& flock) {
    for (const auto& b : flock) b->fly(); // trusts every Bird can fly
}

int main() {
    vector<unique_ptr<Bird>> flock;
    flock.push_back(make_unique<Sparrow>());
    flock.push_back(make_unique<Penguin>());
    try {
        migrate(flock);
    } catch (const exception& e) {
        cout << "CRASH: " << e.what() << "\n";
    }
    return 0;
}
```

OUTPUT

```
Sparrow is flying
CRASH: Penguins can't fly!
```

Bird promises that every bird can **fly()**. **Penguin** cannot keep that promise, so it throws, and the generic **migrate()** function crashes. The fix is to put **fly()** only on birds that actually fly:

C++ · birds_good.cpp

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Bird { // what ALL birds can do
```

C++ · birds_good.cpp

```
public:
    virtual ~Bird() {}
    virtual string name() const = 0;
    void eat() const { cout << name() << " is eating\n"; }
};

class FlyingBird : public Bird {           // only birds that really fly
public:
    void fly() const { cout << name() << " is flying\n"; }
};

class Sparrow : public FlyingBird {
public:
    string name() const override { return "Sparrow"; }
};

class Eagle : public FlyingBird {
public:
    string name() const override { return "Eagle"; }
};

class Penguin : public Bird {
public:
    string name() const override { return "Penguin"; }
    void swim() const { cout << name() << " is swimming\n"; }
};

// migrate() now accepts ONLY birds that can fly: the compiler enforces it
void migrate(const vector<FlyingBird*>& flock) {
    for (auto b : flock) b->fly();
}

int main() {
    Sparrow s;
    Eagle e;
    Penguin p;
    migrate({&s, &e});
    // migrate({&p});           // compile error: a Penguin is not a FlyingBird
    for (const Bird* b : vector<const Bird*>({&s, &e, &p}) b->eat();
    p.swim();
    return 0;
}
```

OUTPUT

```
Sparrow is flying
Eagle is flying
Sparrow is eating
Eagle is eating
Penguin is eating
Penguin is swimming
```

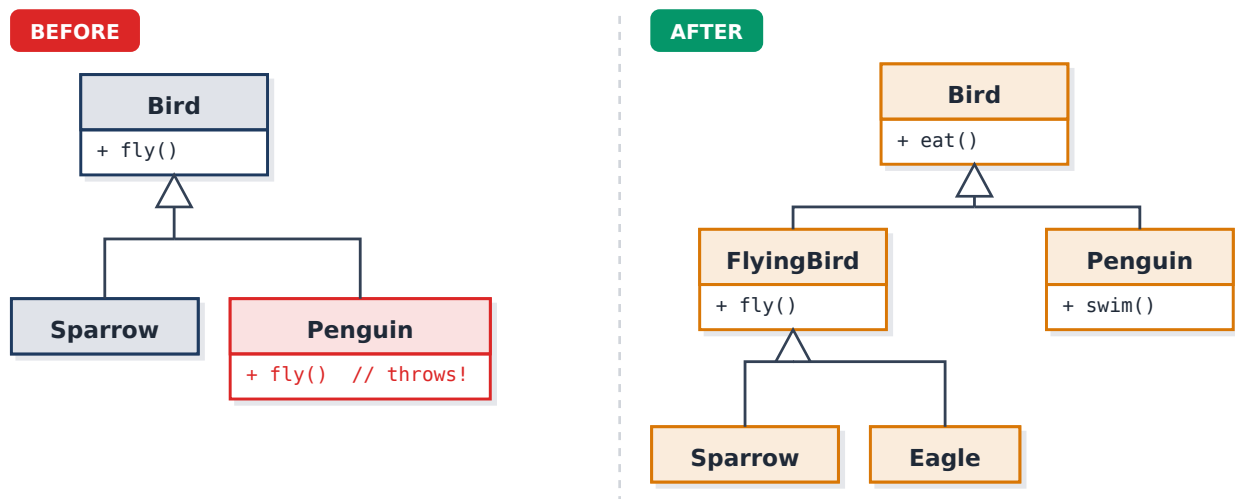


Figure 4.3 Moving fly() down into FlyingBird.

Now the **compiler** prevents the mistake: passing a **Penguin** to **migrate()** is a compile error instead of a run-time crash.

4.5 The rules of substitutability

LSP is often explained as **design by contract**. Each method has a contract: what it needs (preconditions), what it promises (postconditions), and what always stays true about the object (invariants). An overriding method must respect the base class's contract:

Rule	Meaning	Violation example
Preconditions cannot be strengthened	The override must accept everything the base method accepts	Base withdraw() accepts any amount up to the balance; the override rejects amounts over Rs 500
Postconditions cannot be weakened	The override must deliver at least what the base promised	Base sort() promises sorted output; the override sorts only the first half
Invariants must be preserved	What is always true for the base must stay true	Rectangle's width and height are independent; Square ties them together
No new unexpected exceptions	Do not throw where the base never throws	Penguin::fly() throws
History constraint	Do not allow state changes the base forbids	An immutable base class with a subclass that adds setters

4.6 How to spot LSP violations

- An override that **throws** "not supported" or does nothing (an empty body).
- Callers that check the type: `if (dynamic_cast<Square*>(r))` before using a base pointer.
- Subclass documentation that says "do not call X on this class".
- Tests written for the base class that fail when run against a subclass.

Tip

A practical test: **run the base class's unit tests against every subclass**. If a subclass fails any of them, it violates LSP.

LSP recap

- A subclass must be usable anywhere the base class is, with no surprises.
- "Is-a" must hold for **behaviour**, not just in everyday language.
- Keep the contract: no stronger preconditions, no weaker postconditions, keep invariants, no surprise exceptions.
- Fix violations by restructuring the hierarchy, often into siblings under a smaller abstraction.

5. Interface Segregation Principle

"Clients should not be forced to depend upon interfaces that they do not use."

Robert C. Martin, who developed the idea while consulting on printer software at Xerox in the 1990s.

5.1 What it means

An interface should be shaped around what its **clients** need, not around everything an implementation could do. Many small, focused interfaces (**role interfaces**) are better than one large one, because:

- Implementers are not forced to write methods they cannot support.
- Clients only see, and only depend on, the methods they actually call.
- In C++, a change to a method in a header recompiles every file that includes it. Fat interfaces mean a change for one client forces recompiling and re-testing all of them.

Real-life analogy

A restaurant with one giant menu for breakfast, lunch, dinner, drinks and desserts forces every customer to read through everything. Separate menus let each customer see only what is relevant. A TV remote with 60 buttons, when you only ever use 6, is a fat interface.

5.2 The problem

C++ · isp_bad.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

// A "fat" interface: every machine must implement everything
class Machine {
public:
    virtual ~Machine() {}
    virtual void print(const string& doc) = 0;
    virtual void scan(const string& doc) = 0;
    virtual void fax(const string& doc) = 0;
};

class MultiFunctionPrinter : public Machine {
public:
    void print(const string& d) override { cout << "Printing " << d << "\n"; }
    void scan(const string& d) override { cout << "Scanning " << d << "\n"; }
    void fax(const string& d) override { cout << "Faxing " << d << "\n"; }
};

// A cheap printer is FORCED to implement scan() and fax()
class BasicPrinter : public Machine {
public:
    void print(const string& d) override { cout << "Printing " << d << "\n"; }
    void scan(const string&) override {
        throw runtime_error("BasicPrinter cannot scan");
    }
    void fax(const string&) override {
        throw runtime_error("BasicPrinter cannot fax");
    }
};

int main() {
    BasicPrinter p;
    Machine& m = p; // looks like a full Machine to the caller...
    m.print("report.pdf");
    try {
        m.scan("photo.jpg"); // ...but blows up at run time
    } catch (const exception& e) {
        cout << "ERROR: " << e.what() << "\n";
    }
    return 0;
}
```

OUTPUT

```
Printing report.pdf
ERROR: BasicPrinter cannot scan
```

BEFORE: one fat interface

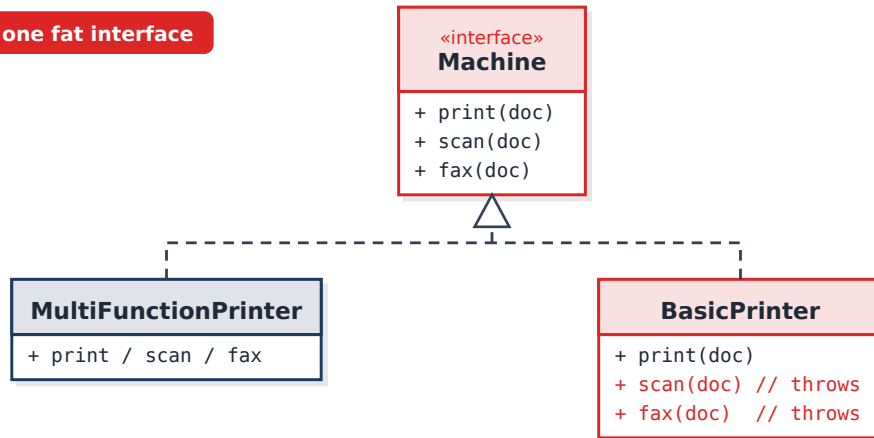


Figure 5.1 BasicPrinter is forced to implement `scan()` and `fax()`.

What goes wrong

- **BasicPrinter** must implement `scan()` and `fax()` even though it cannot do either, so it throws. This is also an **LSP violation**: it cannot be used everywhere a **Machine** is expected.
- The compiler cannot help: `m.scan()` compiles, and fails only at run time.
- Code that only prints still depends on scanning and faxing. Changing `fax()`'s signature forces changes in every printer class, even ones that never fax.

5.3 The fix: small role interfaces

C++ · isp_good.cpp

```
#include <iostream>
#include <string>
using namespace std;

// Small, focused interfaces ("role interfaces")
class Printer {
public:
    virtual ~Printer() {}
    virtual void print(const string& doc) = 0;
};

class Scanner {
public:
    virtual ~Scanner() {}
    virtual void scan(const string& doc) = 0;
};

class FaxMachine {
public:
    virtual ~FaxMachine() {}
    virtual void fax(const string& doc) = 0;
};

// Implements only what it can really do
class BasicPrinter : public Printer {
public:
    void print(const string& d) override { cout << "[Basic] Printing " << d << "\n"; }
};

// Combines several roles through multiple inheritance
class MultiFunctionPrinter : public Printer, public Scanner, public FaxMachine {
public:
    void print(const string& d) override { cout << "[MFP] Printing " << d << "\n"; }
    void scan(const string& d) override { cout << "[MFP] Scanning " << d << "\n"; }
    void fax(const string& d) override { cout << "[MFP] Faxing " << d << "\n"; }
};

// Each client depends ONLY on the role it needs
void printReport(Printer& p) { p.print("report.pdf"); }
void digitize(Scanner& s) { s.scan("old-photo.jpg"); }

int main() {
    BasicPrinter basic;
    MultiFunctionPrinter mfp;

    printReport(basic);
    printReport(mfp);
    digitize(mfp);
    // digitize(basic); // compile error: BasicPrinter is not a Scanner
    return 0;
}
```

OUTPUT

```
[Basic] Printing report.pdf
[MFP] Printing report.pdf
[MFP] Scanning old-photo.jpg
```

AFTER: small role interfaces

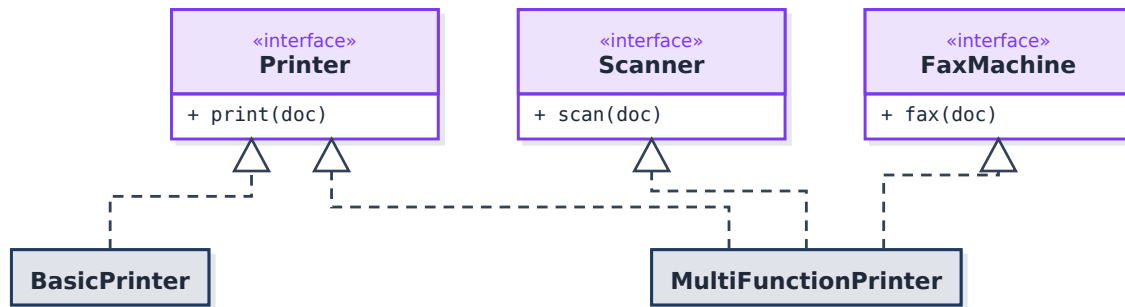


Figure 5.2 Each device implements only the roles it really supports.

`BasicPrinter` now implements only `Printer`. The multifunction device combines all three roles using C++ multiple inheritance of interfaces. Each client function asks for exactly the role it needs, so `digitize(basic)` is rejected by the **compiler**, not discovered by a crash.

5.4 How to spot ISP violations

- Methods implemented as empty bodies, `return false;` or `throw "not supported"`.
- Interfaces with words like `All`, `Manager` or many unrelated method groups.
- Clients that use only one or two methods of a large interface.
- One interface changing frequently because of requests from different clients.

Note

ISP does not mean every interface must have one method. Group methods that are **always used together** by the same clients. `Printer` could reasonably have both `print()` and `cancelJob()`.

ISP recap

- Do not force classes to depend on methods they do not use.
- Split fat interfaces into small role interfaces shaped around client needs.
- A class can implement several roles (multiple inheritance of interfaces in C++).
- ISP violations often cause LSP violations (throwing "not supported").

6. Dependency Inversion Principle

"High-level modules should not depend on low-level modules. Both should depend on abstractions." "Abstractions should not depend on details. Details should depend on abstractions."

Robert C. Martin (1996).

6.1 What it means

High-level modules contain the important business policy: *when an order ships, notify the customer*. **Low-level modules** are the details of how things get done: sending email through a particular service, writing to a particular database. Details change far more often than policy.

Without DIP, the high-level code directly uses the low-level classes, so it depends on them and changes whenever they change. DIP **inverts** that direction: the high-level module defines an interface describing what it needs, and the low-level modules **implement** that interface. Now both depend on the abstraction, and the detail can be swapped without touching the policy.

Real-life analogy

A lamp does not have wires soldered into the power station. It depends on a standard socket (the abstraction). The power can come from the grid, solar panels or a generator (the details), and the lamp never changes.

A good manager says "I need this report by Friday" (an abstraction), not "Ravi must type it on his laptop" (a concrete detail). Anyone capable can do it.

6.2 The problem

C++ · dip_bad.cpp

```
#include <iostream>
#include <string>
using namespace std;

// Low-level detail
class EmailSender {
public:
    void send(const string& to, const string& msg) {
        cout << "[EMAIL to " << to << "]" << msg << "\n";
    }
};

// High-level business policy ... welded to one specific detail
class OrderNotifier {
    EmailSender email; // creates and owns a concrete class
public:
    void orderShipped(const string& customer) {
        email.send(customer, "Your order has shipped!");
    }
};

int main() {
    OrderNotifier notifier;
    notifier.orderShipped("riya@example.com");
    // Want SMS instead? WhatsApp? A fake sender for unit tests?
    // Every option requires editing OrderNotifier itself.
    return 0;
}
```

OUTPUT

[EMAIL to riya@example.com] Your order has shipped!

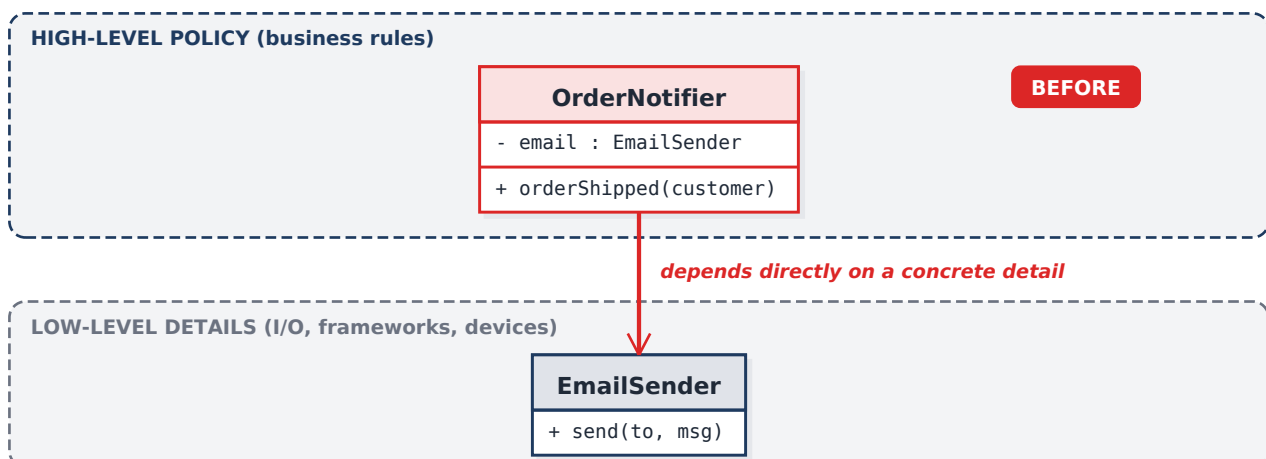


Figure 6.1 The business policy depends directly on a low-level detail.

What goes wrong

- Switching to SMS or WhatsApp means editing **OrderNotifier**, the business logic.
- It cannot be unit-tested** without actually sending an email, because the email sender is created inside the class.
- Any change to **EmailSender** (its constructor, its API) ripples upward into the business layer.

6.3 The fix: depend on an abstraction, inject the detail

C++ · dip_good.cpp

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// The abstraction: owned by the high-level side, describes what IT needs
class MessageSender {
public:
    virtual ~MessageSender() {}
    virtual void send(const string& to, const string& msg) = 0;
};

// High-level policy depends ONLY on the abstraction
class OrderNotifier {
    MessageSender& sender;
public:
    explicit OrderNotifier(MessageSender& s) : sender(s) {} // injected
    void orderShipped(const string& customer) {
        sender.send(customer, "Your order has shipped!");
    }
};

// Low-level details also depend on the abstraction (they implement it)
class EmailSender : public MessageSender {
public:
    void send(const string& to, const string& msg) override {
        cout << "[EMAIL to " << to << "] " << msg << "\n";
    }
};

class SmsSender : public MessageSender {
public:
    void send(const string& to, const string& msg) override {
        cout << "[SMS to " << to << "] " << msg << "\n";
    }
};

// A fake for unit tests: records messages instead of sending them
class FakeSender : public MessageSender {
public:
    vector<string> sent;
    void send(const string& to, const string& msg) override {
        sent.push_back(to + ": " + msg);
    }
};

int main() {
    EmailSender email;
    SmsSender sms;
    OrderNotifier(email).orderShipped("riya@example.com");
    OrderNotifier(sms).orderShipped("+91-98xxxxxx10");

    // Unit test: no network, no real email, fully predictable
    FakeSender fake;
    OrderNotifier(fake).orderShipped("test-user");
    bool ok = fake.sent.size() == 1 &&
        fake.sent[0] == "test-user: Your order has shipped!";
    cout << "Unit test: " << (ok ? "PASSED" : "FAILED") << "\n";
    return 0;
}
```

OUTPUT

```
[EMAIL to riya@example.com] Your order has shipped!  
[SMS to +91-98xxxxxx10] Your order has shipped!  
Unit test: PASSED
```

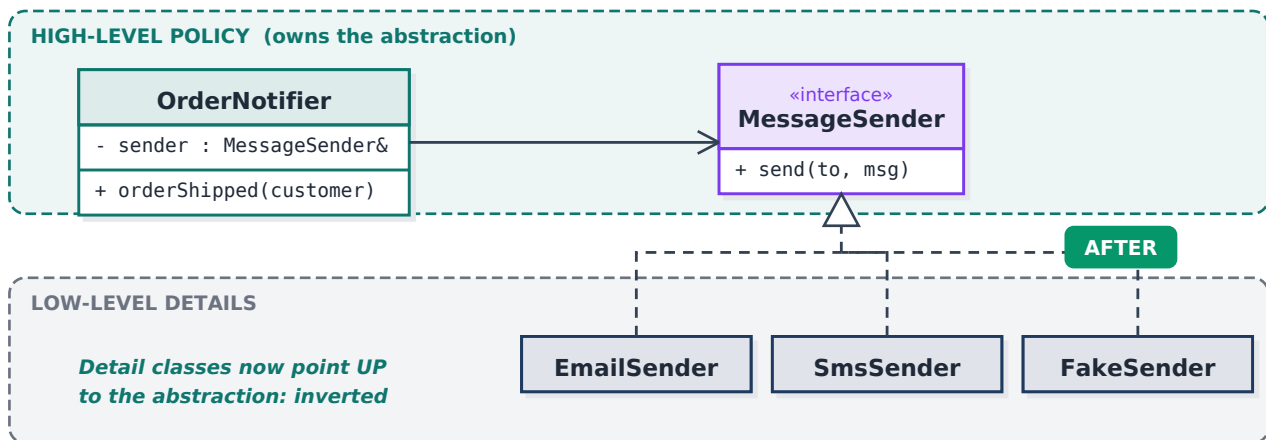


Figure 6.2 The dependency arrows from the detail classes now point upward to the abstraction: the dependency is inverted.

Three things changed:

1. `MessageSender` is an interface **defined for the needs of the high-level code**. It belongs with the business logic, not with the email library.
2. `OrderNotifier` no longer creates its sender; it **receives** one through its constructor. This is called **dependency injection**.
3. Swapping email for SMS, or for a `FakeSender` in a unit test, requires **no change** to `OrderNotifier`. The test at the end of `main()` runs in microseconds without a network.

6.4 DIP, dependency injection and inversion of control

These three terms are related but different, and are often confused in interviews:

Term	What it is	In the example
Dependency Inversion Principle	A design principle: depend on abstractions, and let details depend on them too	<code>OrderNotifier</code> depends on <code>MessageSender</code>
Dependency Injection (DI)	A technique: pass dependencies in from outside (constructor, setter or parameter) instead of creating them inside	<code>OrderNotifier(MessageSender& s)</code>
Inversion of Control (IoC)	A broader idea: a framework or outer code controls creating and wiring objects, not the object itself	<code>main()</code> decides which sender to plug in

6.5 DIP with templates

In C++ you can also invert dependencies at compile time. The template parameter plays the role of the abstraction; there are no virtual calls, at the cost of choosing the detail at compile time:

C++ · dip_template.cpp

```
#include <iostream>
#include <string>
using namespace std;

// DIP with templates: the "interface" is whatever Sender provides.
// No virtual calls; checked at compile time.
template <typename Sender>
class OrderNotifier {
    Sender& sender;
public:
    explicit OrderNotifier(Sender& s) : sender(s) {}
    void orderShipped(const string& customer) {
        sender.send(customer, "Your order has shipped!");
    }
};

struct ConsoleSender {
    void send(const string& to, const string& msg) {
        cout << "[CONSOLE " << to << " ] " << msg << "\n";
    }
};

int main() {
    ConsoleSender c;
    OrderNotifier<ConsoleSender> n(c);
    n.orderShipped("aarav");
    return 0;
}
```

OUTPUT

```
[CONSOLE aarav] Your order has shipped!
```

6.6 How to spot DIP violations

- Business classes that **#include** database, network, file or UI headers.
- **new ConcreteClass** or concrete member objects for things that might change (storage, messaging, payment, time).
- Classes that are hard to unit-test because they talk to real external systems.
- Changing a library or vendor requires editing business logic.

Note

Not every dependency needs inverting. Depending directly on **stable** things such as `std::string`, `std::vector` or your own value types is fine. Invert dependencies on things that are **volatile** or **external**: databases, networks, file systems, clocks, third-party services.

DIP recap

- High-level policy and low-level details should both depend on abstractions.
- The high-level side owns the interface; details implement it.
- Inject dependencies (usually through the constructor) instead of creating them.
- Main payoffs: swappable details and easy unit testing.

7. Putting It All Together: Refactoring an Order System

Real code rarely breaks just one principle. This chapter takes one realistic "God class" and refactors it step by step until all five principles are satisfied.

7.1 Before

C++ · project_bad.cpp

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Order {
    int id;
    string customerType;    // "regular", "premium", "vip"
    string email;
    vector<double> prices;
};

// The "God class": does everything, depends on everything
class OrderProcessor {
public:
    void process(const Order& o) {
        // validation
        if (o.prices.empty()) { cout << "Order " << o.id << " invalid\n"; return; }

        // pricing with a type switch
        double total = 0;
        for (double p : o.prices) total += p;
        if (o.customerType == "premium") total *= 0.90;
        else if (o.customerType == "vip") total *= 0.80;

        // hard-wired database code
        cout << "INSERT INTO orders VALUES(" << o.id << ", " << total << ")\n";

        // hard-wired email code
        cout << "SMTP: mail " << o.email << " 'Order " << o.id
            << " confirmed, Rs " << total << "\n";
    }
};

int main() {
    OrderProcessor p;
    p.process({1, "vip", "riya@example.com", {500, 1500}});
    p.process({2, "regular", "aarav@example.com", {}});
    return 0;
}
```

OUTPUT

```
INSERT INTO orders VALUES(1, 1600)
SMTP: mail riya@example.com 'Order 1 confirmed, Rs 1600'
Order 2 invalid
```

Problem in OrderProcessor	Principle broken
Validation, pricing, saving and emailing in one method	SRP : four reasons to change
<code>if (customerType == ...)</code> chain for discounts	OCP : new type = edit this class
Hard-wired SQL and SMTP code	DIP : business logic depends on details
Impossible to test without a database and mail server	DIP (and SRP)
Any future storage interface likely to grow into a fat "do everything" class	ISP risk

7.2 After

C++ · project_good.cpp

```
#include <iostream>
#include <map>
#include <memory>
#include <string>
#include <vector>
using namespace std;

struct Order {
    int id;
    string email;
    vector<double> prices;
};

// ----- SRP: one small class per responsibility -----
class OrderValidator {
public:
    bool isValid(const Order& o) const { return !o.prices.empty(); }
};

// ----- OCP + LSP: pricing rules are pluggable and substitutable -----
class DiscountPolicy {
public:
    virtual ~DiscountPolicy() {}
    virtual double apply(double total) const = 0;    // must return 0..total
};
class NoDiscount : public DiscountPolicy {
public:
    double apply(double t) const override { return t; }
};
class PercentDiscount : public DiscountPolicy {
    double pct;
public:
    explicit PercentDiscount(double p) : pct(p) {}
    double apply(double t) const override { return t * (1 - pct / 100); }
};

// ----- ISP: small interfaces, one role each -----
class OrderRepository {
public:
    virtual ~OrderRepository() {}
    virtual void save(int id, double total) = 0;
};
class Notifier {
public:
    virtual ~Notifier() {}
    virtual void notify(const string& to, const string& msg) = 0;
};

// ----- DIP: high-level service depends only on abstractions -----
class OrderService {
    OrderValidator validator;
    const DiscountPolicy& discount;
    OrderRepository& repo;
    Notifier& notifier;
public:
    OrderService(const DiscountPolicy& d, OrderRepository& r, Notifier& n)
        : discount(d), repo(r), notifier(n) {}

    bool place(const Order& o) {
        if (!validator.isValid(o)) {
            cout << "Order " << o.id << " rejected: no items\n";
        }
    }
};
```

C++ · project_good.cpp

```

        return false;
    }
    double total = 0;
    for (double p : o.prices) total += p;
    total = discount.apply(total);
    repo.save(o.id, total);
    notifier.notify(o.email, "Order " + to_string(o.id) + " confirmed");
    return true;
}
};

// ----- Low-level details, plugged in from outside -----
class MySqlRepo : public OrderRepository {
public:
    void save(int id, double total) override {
        cout << " [MySQL] INSERT order " << id << " total " << total << "\n";
    }
};

class MemoryRepo : public OrderRepository {
public:
    map<int, double> rows;
    void save(int id, double total) override { rows[id] = total; }
};

class EmailNotifier : public Notifier {
public:
    void notify(const string& to, const string& msg) override {
        cout << " [Email] to " << to << ": " << msg << "\n";
    }
};

class SilentNotifier : public Notifier {
public:
    int count = 0;
    void notify(const string&, const string&) override { ++count; }
};

int main() {
    // Production wiring
    PercentDiscount vip(20);
    MySqlRepo db;
    EmailNotifier email;
    OrderService shop(vip, db, email);
    shop.place({1, "riya@example.com", {500, 1500}});
    shop.place({2, "aarav@example.com", {}});

    // Test wiring: same OrderService, fake details, no database or email
    NoDiscount none;
    MemoryRepo memDb;
    SilentNotifier quiet;
    OrderService testShop(none, memDb, quiet);
    testShop.place({7, "t@test", {100, 200}});
    cout << "Test: saved total = " << memDb.rows[7]
        << ", notifications = " << quiet.count << "\n";
    return 0;
}

```

OUTPUT

```

[MySQL] INSERT order 1 total 1600
[Email] to riya@example.com: Order 1 confirmed
Order 2 rejected: no items
Test: saved total = 300, notifications = 1

```

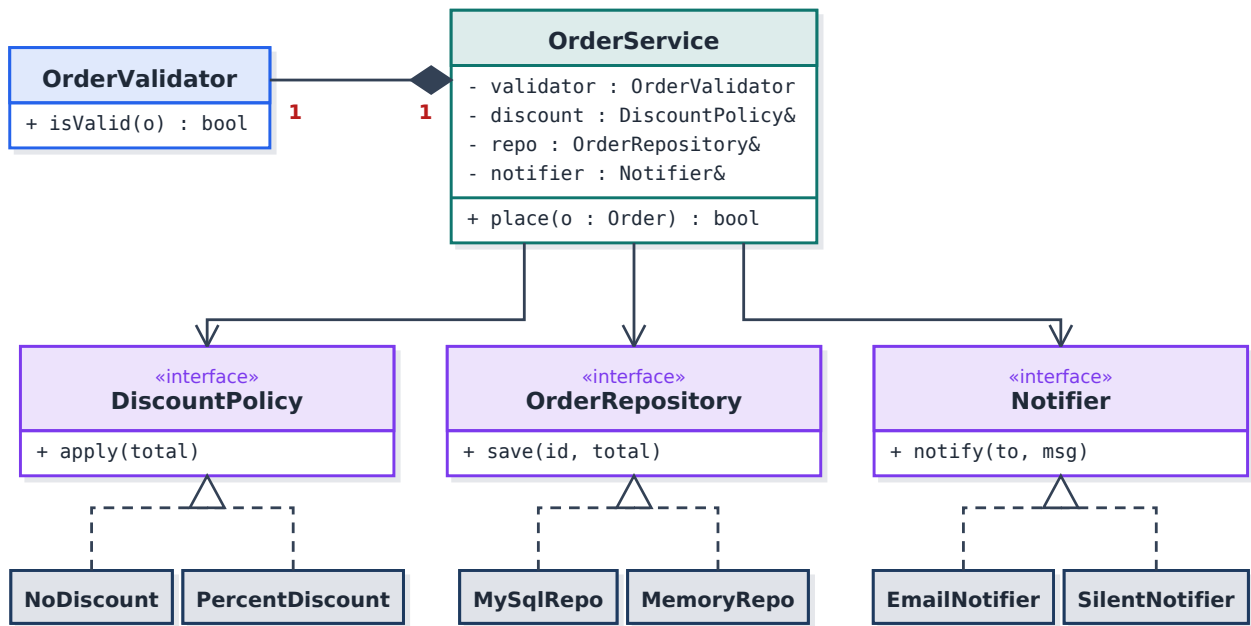


Figure 7.1 The refactored design: OrderService depends only on small abstractions.

7.3 Where each principle appears

Principle	Applied by
SRP	Validation moved into <code>OrderValidator</code> ; pricing, storage and notification each behind their own abstraction. <code>OrderService</code> only coordinates.
OCP	New discount schemes are new <code>DiscountPolicy</code> classes. <code>OrderService</code> is never edited for them.
LSP	Every <code>DiscountPolicy</code> keeps the contract "return a value between 0 and total", so any policy can be substituted safely. The comment on <code>apply()</code> records the contract.
ISP	<code>OrderRepository</code> and <code>Notifier</code> are tiny role interfaces; <code>SilentNotifier</code> implements one method, not a whole messaging API.
DIP	<code>OrderService</code> receives all its dependencies through the constructor. <code>main()</code> wires in MySQL and email for production, and in-memory fakes for the test.

Why this matters

The final lines of output show the real payoff. The exact same `OrderService` code ran against a fake repository and a silent notifier, and the test checked the saved total (300) and the notification count (1), with no database or mail server involved. Code designed with SOLID is **testable by construction**.

8. Using SOLID Wisely

SOLID principles are guidelines for managing change, not laws. Applied blindly, they create the last design smell from Chapter 1: **needless complexity**.

8.1 Signs you have gone too far

- An interface for every class, each with exactly one implementation that never changes.
- Ten files to follow to understand a feature that could be 30 clear lines.
- Abstractions for changes nobody has asked for ("in case we switch databases one day").
- New team members cannot find where the actual work happens.

8.2 Principles that balance SOLID

Principle	Meaning	How it balances SOLID
KISS	Keep it simple	Prefer the simplest design that works today
YAGNI	You aren't gonna need it	Do not add abstractions for imagined future needs
DRY	Don't repeat yourself	Duplicated type-switches are a real signal to apply OCP
Rule of three	Refactor when you see the same kind of change the third time	Tells you <i>when</i> to introduce an abstraction

Tip

A practical approach used by many experienced developers:

- Write the straightforward version first.
- When a change is painful, ask which principle would have made it easy.
- Refactor toward that principle then, with tests protecting you.

SOLID is most valuable at the boundaries that change often: business rules, storage, external services and user interfaces.

8.3 SOLID and design patterns

Many classic design patterns are ready-made ways of applying SOLID:

Pattern	What it does	Principles
Strategy	Encapsulate interchangeable algorithms behind an interface (<code>DiscountPolicy</code>)	OCP, DIP
Template Method	Fixed algorithm skeleton with overridable steps	OCP
Decorator	Add behaviour by wrapping an object instead of editing it	OCP, SRP
Adapter	Make an external class fit your own interface	DIP, ISP

Pattern	What it does	Principles
Factory	Centralise object creation so clients depend only on interfaces	DIP, OCP
Observer	Notify any number of listeners without knowing their types	OCP, DIP
Facade	Simple interface over a complex subsystem	ISP, SRP

9. Interview Questions and Answers

Q1. What does SOLID stand for?

Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion.

Q2. Why do we need SOLID principles?

To keep code easy to change, test, reuse and understand as it grows, by keeping cohesion high and coupling low.

Q3. Explain SRP with an example.

A class should have one reason to change. An Invoice that calculates, prints, saves and emails should be split so tax rules, layout, storage and messaging change independently.

Q4. Is SRP the same as 'a class should do one thing'?

Not exactly. It means one reason to change (one actor). A class can have several methods that all serve the same responsibility.

Q5. Explain OCP. How do you achieve it in C++?

Code should be extendable without modifying it. Use abstractions: interfaces with virtual functions, templates, or callables, so new behaviour is added as new classes.

Q6. Give a typical sign of an OCP violation.

An if/else or switch on a type code that must be edited every time a new type is added.

Q7. Explain LSP. Why is Square/Rectangle a violation?

Subtypes must be substitutable for their base types. A mutable Square changes both sides in setWidth/setHeight, so code relying on independent width and height breaks.

Q8. What are the contract rules of LSP?

Do not strengthen preconditions, do not weaken postconditions, preserve invariants, and do not throw new unexpected exceptions.

Q9. Explain ISP with an example.

Do not force classes to implement methods they do not use. Split a Machine interface into Printer, Scanner and Fax so a basic printer implements only Printer.

Q10. How are ISP and LSP related?

A fat interface often forces implementers to throw 'not supported', which makes them non-substitutable, violating LSP.

Q11. Explain DIP.

High-level modules and low-level modules should both depend on abstractions; details depend on abstractions, not the other way round.

Q12. What is the difference between DIP and dependency injection?

DIP is the principle (depend on abstractions). Dependency injection is a technique for supplying the concrete implementation from outside, e.g. via the constructor.

Q13. Who should own the interface in DIP?

The high-level (client) side, because the interface describes what the high-level policy needs.

Q14. How does SOLID make code testable?

With DIP, real dependencies like databases can be replaced by fakes; with SRP, each class is small and focused, so tests are simple.

Q15. Can SOLID be overused?

Yes. Excessive abstraction adds needless complexity. Balance it with KISS and YAGNI and introduce abstractions when change actually demands them.

Q16. Which design patterns support OCP?

Strategy, Template Method, Decorator, Observer and Factory, among others.

Q17. Does DIP mean never using concrete classes?

No. Depending on stable things (standard library, value types) is fine. Invert dependencies on volatile or external details.

10. Practice Exercises

Each exercise gives deliberately badly designed code. Identify which principle it breaks, explain the concrete problem it will cause, then refactor it and write a small `main()` that proves your version works.

1. Sales report

C++ · `ex_report.cpp`

```
class SalesReport {
public:
    void loadFromDatabase(const string& query);    // talks to the DB
    double calculateTotals();                     // business maths
    string toHtml();                             // formatting
    string toCsv();                              // formatting
    void uploadToS3(const string& bucket);        // cloud storage
};
```

Task: Which principle? List the reasons this class could change and who would ask for each. Split it accordingly.

Hint: Look for four groups: data access, calculation, formatting, storage.

2. Shipping cost

C++ · `ex_shipping.cpp`

```
double shippingCost(const string& method, double weightKg) {
    if (method == "standard")    return 40 + 10 * weightKg;
    else if (method == "express") return 90 + 15 * weightKg;
    else if (method == "pickup") return 0;
    return -1;                  // unknown method
}
```

Task: Add a new "same-day" method without editing any existing function after your refactoring.

Hint: Create a `ShippingMethod` interface with `cost(weight)`.

3. Read-only file

C++ · ex_readonly.cpp

```
class File {
public:
    virtual ~File() {}
    virtual string read() { return "data"; }
    virtual void write(const string& s) { /* save s */ }
};

class ReadOnlyFile : public File {
public:
    void write(const string&) override {
        throw runtime_error("read-only!");
    }
};
```

Task: Why is `ReadOnlyFile` a problem for code that receives a `File&`? Redesign the hierarchy.

Hint: Separate `Readable` and `Writable` roles; this combines LSP and ISP.

4. Robot workers

C++ · ex_worker.cpp

```
class Worker {
public:
    virtual ~Worker() {}
    virtual void work() = 0;
    virtual void eat() = 0;
    virtual void sleep() = 0;
};

class Robot : public Worker {
public:
    void work() override { /* assemble parts */ }
    void eat() override { } // robots don't eat...
    void sleep() override { } // ...or sleep
};
```

Task: Robots are forced to implement `eat()` and `sleep()`. Fix the interface design.

Hint: Split into `Workable`, `Eatable`, `Sleepable`; a human implements all three.

5. Logger

C++ · ex_logger.cpp

```
class FileLogger {
public:
    void log(const string& msg) { /* append to app.log */ }
};

class PaymentService {
    FileLogger logger;           // concrete dependency
public:
    void pay(double amount) {
        logger.log("Paid " + to_string(amount));
    }
};
```

Task: Make `PaymentService` testable without writing to a real file, and able to log to the console or a remote server.

Hint: Introduce a `Logger` interface and inject it through the constructor.

Design challenges

1. **Notification hub.** Design a system that sends alerts by email, SMS and push notification, where new channels can be added without changing existing code and the core logic can be unit tested. Draw the UML class diagram, then implement it.
2. **Parking lot.** Model cars, bikes and trucks with different parking fees. Make sure adding an electric-car charging fee requires no change to the fee-calculation code.
3. **Audit the God class.** Take the largest class from one of your own past projects. List every principle it breaks and sketch a refactored design.

11. Cheat Sheet and Glossary

	Rule	Smell	Typical fix in C++
S	One reason to change	Class described with "and"; mixed includes	Split into focused classes
O	Extend without modifying	<code>if/switch</code> on type codes	Interface + subclasses, templates, <code>std::function</code>
L	Subtypes keep base promises	Overrides that throw or do nothing; <code>dynamic_cast</code> checks	Restructure hierarchy; smaller base abstraction
I	No forced unused methods	Fat interfaces; "not supported" methods	Split into role interfaces; multiple inheritance
D	Depend on abstractions	<code>new</code> of concrete details; untestable classes	Interface owned by high level; constructor injection

Glossary

Term	Meaning
Abstraction	An interface or abstract class describing what something does, not how.
Actor (SRP)	A person or group that can request changes to a module.
Cohesion	How closely the contents of a class belong together. High is good.
Contract	A method's preconditions, postconditions and invariants.
Coupling	How much one class depends on others. Low is good.
Dependency injection	Supplying a class's dependencies from outside, e.g. via its constructor.
Design smell	A symptom of poor design: rigidity, fragility, immobility, viscosity.
God class	A class that knows and does far too much.
High-level module	Code containing business policy and rules.
Invariant	A condition that is always true for an object.
Low-level module	Code handling details: I/O, databases, networks, frameworks.
Postcondition	What a method guarantees when it finishes.
Precondition	What must be true before a method may be called.
Role interface	A small interface describing one role a class can play.
Substitutability	The ability to use a subtype wherever its base type is expected.
YAGNI	"You aren't gonna need it": avoid building for imagined future needs.